## Introduction

In any programming language, one must trade off readability and maintainability on the one hand, and performance on the other hand. One common source of this tension is the need to access objects related to a given one. [1]

```
# The arithmetic of the interpreted language: what a value is, what a
# literal means, and how to print one.
type Value;

assume Std, Value {
  fn value(n: Int): Value;
  fn print_value(v: Value);

  context Arith = Value, value, print_value, Value.add, Value.sub, Value.mul,
Value.div;

  # Called with every intermediate value.
  fn trace(v: Value);
}

functor Quiet: Std, Value -> trace {
  bind trace = (v) => {};
}

functor Loud: Std, Arith -> trace {
  bind trace = (v) => {
    putchar(char::space);
    putchar(char::space);
    print_value(v);
  };
}

# Written once, against the interfaces alone.
assume Std, Ast, Arith, trace {
  fn interp(e: ExprRef): Value {
    let v = match e.data() {
      Binary { op, lhs, rhs } => {
        let l = interp(lhs);
        let r = interp(rhs);
        match op {
          Add => l + r,
          Sub => l - r,
          Mul => l * r,
          Div => l / r,
        }
      },
      Literal n => value(n),
      Traced inner => {
        bind Loud;
        interp(inner)
      },
    };
    trace(v);
    v
  }
```

```
  fn report(e: ExprRef) { print_value(interp(e)); }
}
```

The most *convenient* way to write this code would be if `foo` had a field called `bar`. But this causes problems if there are a *million* `Foo`s.

# Prior work

Lisp

Koka

Zig

# Completed work

## Theory

(syntax figure)

(static semantics figure)

(dynamic semantics figure)

## Implementation

We have implemented a compiler from Moss to WebAssembly (TODO citation) (Wasm), written in Python. This implementation supports the full language described in the previous section, plus the following additional language features:

- TODO

We have also implemented a self-hosting compiler from Moss to Wasm, which can be bootstrapped from the Python implementation:

- The Python compiler is Tier 0.
- Tier 0 can be run on the Moss compiler source code to produce the Tier 1 compiler.
- The Tier 1 compiler can be run on the Moss compiler source code to produce the Tier 2 compiler.
- The Tier 2 compiler can be run on the Moss compiler source code to produce the Tier 3 compiler.

The Wasm binaries for the Tier 2 and Tier 3 compilers are bytewise identical.

# Proposed work

## Performance experiments

# Bibliography

[1] A. Sampson, “Flattening ASTs (and Other Compiler Data Structures).” [Online]. Available: https://www.cs.cornell.edu/~asampson/blog/flattening.html